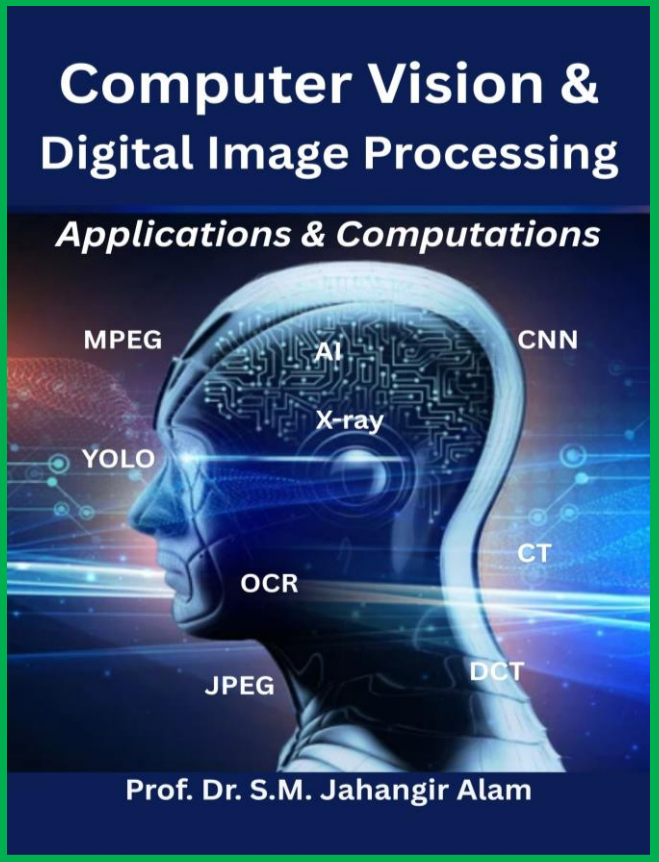
# Lecture 8.1

# Image Representation: Morphological Processing




Instructor Name:

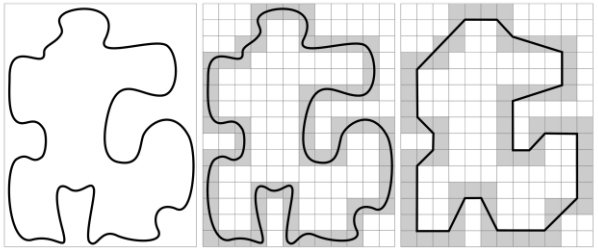
**Prof. Dr. Jahangir Alam**

**Dept. of CSE, BAUST**

# Morphological Operations

**Morphological operations** are a set of image processing techniques used to analyze and manipulate the structure or shape of objects in an image. They're widely used in digital image processing and computer vision, especially for binary (black-and-white) images—but they also work on grayscale images.

These operations use a small shape called a **structuring element** (kernel) to probe and transform the image.



**Applications**

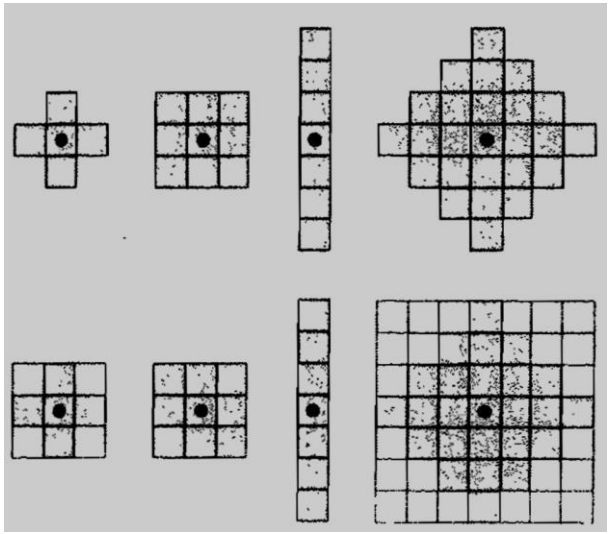
- Object detection and segmentation
- Noise removal in images
- Medical imaging (e.g., tumor detection)
- OCR (text extraction from images)
- Edge and boundary detection

# Structuring Elements (SEs)

A **structuring element** is a small matrix (kernel) used to examine and transform an image in digital image processing.

- **Set Reflection** & **Translation** are employed extensively in morphology to formulate operations based on so-called **Structuring Elements (SEs):**

small set or sub-images used to probe an image under study for properties of interest



**Structure of a Structuring Element**

- A **shape** (square, circle, line, etc.)
- A **size** (e.g., 3×3, 5×5)
- An **origin (anchor point)** — usually the center

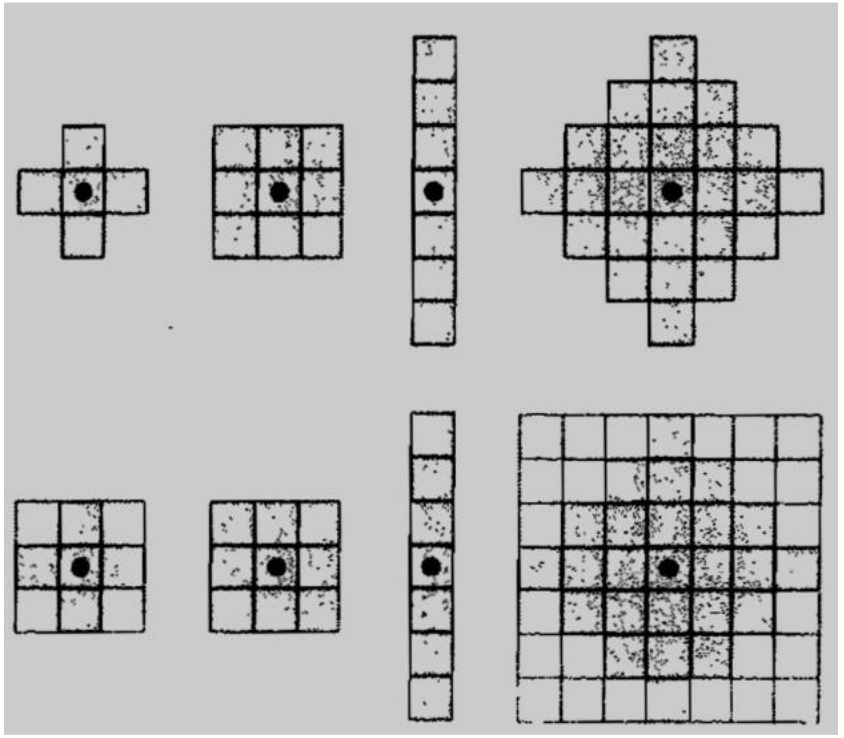
**Example (3×3 square SE):**

$$\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

# Structuring Elements (SEs)

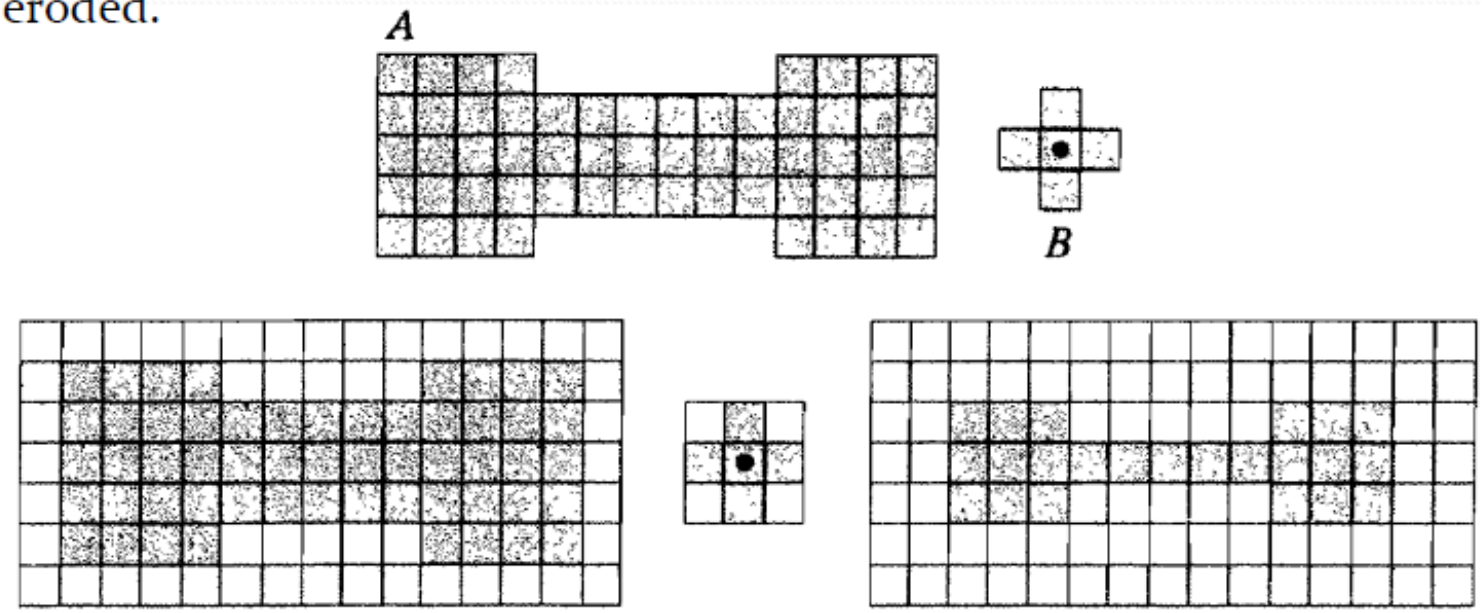
**First row:** Examples of structuring elements.

**Second row:** Structuring elements converted to rectangular arrays. The dots denote the centers of the SEs



# Structuring Elements (SEs)

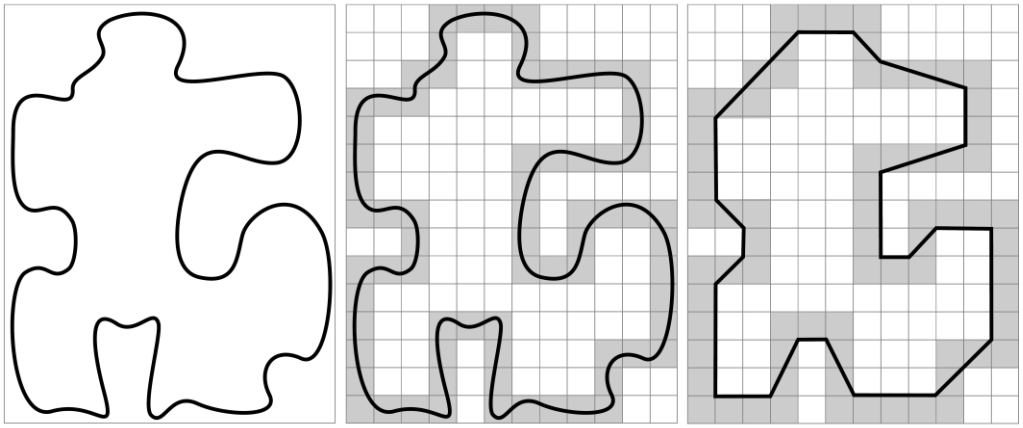
**Example:** Define an operation on set A using SE (B) by running B over A such that the origin of B visits every element of A. The net result is that the boundary of set A is eroded.

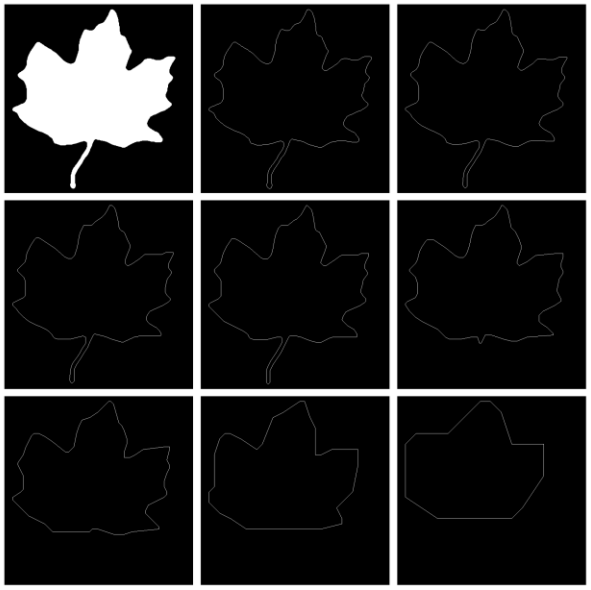



a b
c d e

**FIGURE 9.3** (a) A set (each shaded square is a member of the set). (b) A structuring element. (c) The set padded with background elements to form a rectangular array and provide a background border. (d) Structuring element as a rectangular array. (e) Set processed by the structuring element.

# Image Representation and Description





# Logic Operations Involving Binary Pixels and Images

- Logical operations are **pixel-wise Boolean operations** applied to **binary (or thresholded) images**.
  Each pixel value is treated as:
- **1 (white / foreground / True)**
- **0 (black / background / False)**

- **Used for:**
- Masking or combining binary images
- Segmentation
- Region extraction
- Feature highlighting

# Logic Operations Involving Binary Pixels and Images

## Applications

| Operation | Application |
|---|---|
| AND | Extract overlapping regions (masking) |
| OR | Combine segmented parts |
| XOR | Detect differences between frames |
| NOT | Invert image or background |
| NAND/NOR | Noise suppression or logical filters |

# Logic Operations Involving Binary Pixels and Images

| Operation | Symbol | Output rule (for each pixel pair A, B) |
|---|---|---|
| **AND** | $A \wedge B$ | 1 if both A and B are 1 |
| **OR** | $A \vee B$ | 1 if either A or B is 1 |
| **NOT** | $\neg A$ | Inverts pixel values (1 → 0, 0 → 1) |
| **XOR** | $A \oplus B$ | 1 if A ≠ B |
| **NAND** | $\overline{A \wedge B}$ | 1 if NOT (A AND B) |
| **NOR** | $\overline{A \vee B}$ | 1 if both A and B are 0 |

# Logic Operations Involving Binary Pixels and Images

$$A = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 1 & 0 \end{bmatrix}, B = \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

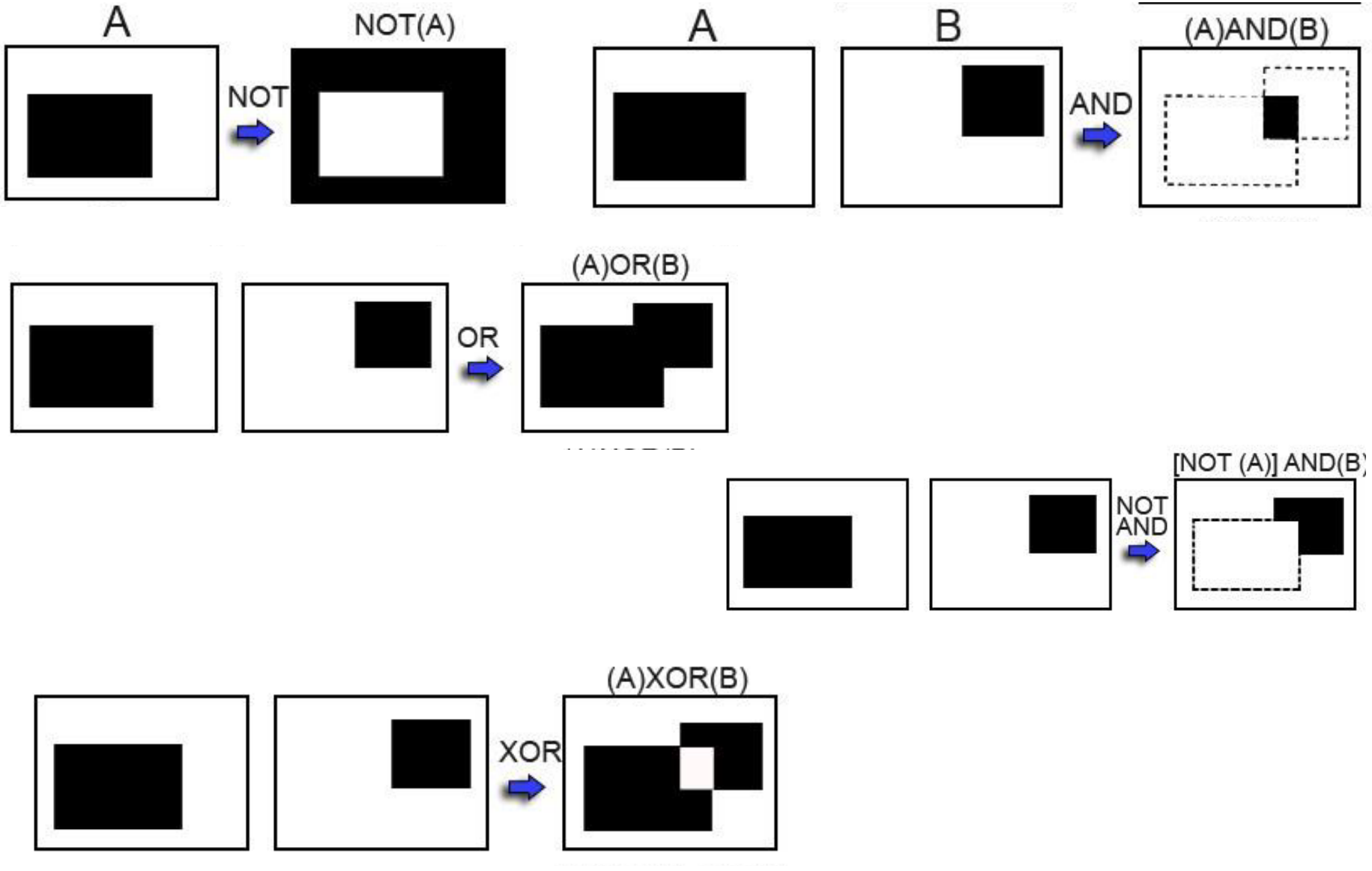
$$A \wedge B = \begin{bmatrix} 1 \wedge 0 & 0 \wedge 1 & 1 \wedge 1 \\ 0 \wedge 1 & 1 \wedge 0 & 0 \wedge 0 \\ 1 \wedge 1 & 1 \wedge 0 & 0 \wedge 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 0 & 0 \\ 1 & 0 & 0 \end{bmatrix}$$

$$A \vee B = \begin{bmatrix} 1 \vee 0 & 0 \vee 1 & 1 \vee 1 \\ 0 \vee 1 & 1 \vee 0 & 0 \vee 0 \\ 1 \vee 1 & 1 \vee 0 & 0 \vee 1 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

$$A \oplus B = \begin{bmatrix} 1 \oplus 0 & 0 \oplus 1 & 1 \oplus 1 \\ 0 \oplus 1 & 1 \oplus 0 & 0 \oplus 0 \\ 1 \oplus 1 & 1 \oplus 0 & 0 \oplus 1 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 0 \\ 1 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix}$$

$$\neg A = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 0 & 1 \end{bmatrix}$$

# Logic Operations Involving Binary Pixels and Images



# Different Morphological Operations

1. **Erosion**
2. **Dilation**
3. **Opening**
4. **Closing**
5. **Hit-or-Miss Transform**
6. **Morphological Gradient**
7. **Top-Hat Transform**
8. **Skeletonization**
9. **Pruning**

- Shrinks objects in an image
- Removes small white noises
- Breaks thin connections
- Effect: boundaries get thinner

Useful for eliminating tiny unwanted details.

- Expands objects
- Fills small holes and gaps
- Connects nearby objects

Opposite of erosion—makes features grow.

- Removes noise (small objects)
- Preserves overall shape

Good for cleaning up images.

# Different Morphological Operations

1. **Erosion**
2. **Dilation**
3. **Opening**
4. **Closing**

- Fills small holes
- Connects broken parts

Useful for smoothing contours and filling gaps.

5. **Hit-or-Miss Transform**
6. **Morphological Gradient**
7. **Top-Hat Transform**
8. **Skeletonization**
9. **Pruning**

# Erosion and Dilation

- Erosion and Dilation are the two most fundamental morphological operations in image processing, primarily used for shape analysis, **noise removal**, and image segmentation (especially for binary or grayscale images).

**In Grayscale Images**

Erosion: Each output pixel = minimum of the neighborhood.

Dilation: Each output pixel = maximum of the neighborhood.

**Applications**

- ✓ Noise removal (opening)
- ✓ Object separation
- ✓ Hole filling (closing)
- ✓ Boundary extraction
- ✓ Skeletonization

# Erosion and Dilation

| Operation | Formula |
|---|---|
| Erosion | $g(x,y) = \min_{(s,t) \in B} f(x+s, y+t)$ |
| Dilation | $g(x,y) = \max_{(s,t) \in B} f(x+s, y+t)$ |

| Operation | Description | Formula |
|---|---|---|
| Opening | Erosion followed by dilation → removes small noise | $A \circ B = (A \ominus B) \oplus B$ |
| Closing | Dilation followed by erosion → fills small holes | $A \cdot B = (A \oplus B) \ominus B$ |

# Erosion

- Erosion is a *morphological operation* that shrinks bright (white) regions and enlarges dark (black) regions in an image. Erosion shrinks the white (foreground) regions of an image.
- It is typically applied to **binary** or **grayscale** images using a small shape called a **structuring element (SE)** (e.g., 3×3 square, cross, or circle).
- If $A$ is the image and $B$ is the structuring element (kernel),

$$A \ominus B = \{z \mid Bz \subseteq A\}$$

# Erosion

- If $A$ is the image and $B$ is the structuring element (kernel),
$$A \ominus B = \{z \mid Bz \subseteq A\}$$
- Slide $B$ over A.
- At each position, check if B "fits" completely inside the white region of $A$.
- Output pixel = **1** only if *all* pixels under $B$ are **1**.
- Slide the structuring element $B$ over the image.
- The output pixel is 1 only if all pixels under $B$ are 1 in the input image.

# Erosion Rule

- For every pixel $(x, y)$,

$$(A \ominus B)(x, y) = \begin{cases} 1 & if\ all\ pixel\ under\ B\ is\ 1 \\ 0 & otherwise \end{cases}$$

**Effect**:

- Removes small white noises.
- Shrinks bright regions.
- Breaks narrow connections between objects.

# Erosion

| Input | Structuring Element (3×3 all 1s) | Output (Eroded) |
|---|---|---|
| 0 0 0 0 0 | 1 1 1 | 0 0 0 0 0 |
| 0 1 1 1 0 | 1 1 1 | 0 0 1 0 0 |
| 0 1 1 1 0 | 1 1 1 | 0 0 1 0 0 |
| 0 0 0 0 0 | 1 1 1 | 0 0 0 0 0 |

**Result:** The “white” region (1s) becomes smaller.

# Dilation

- **Dilation** is a morphological operation that **expands** or **grows** the bright (white) regions in an image.
- Dilation expands the white (foreground) regions of an image.

$$A \oplus B = \{z \mid (Bz \cap A) \neq \emptyset\}$$

- The output pixel is 1 if at least one pixel under the structuring element $B$ is 1.

- **Dilation Rule:** For every pixel $(x, y)$,

$$(A \oplus B)(x, y) = \begin{cases} 1 & if\ any\ pixel\ under\ B\ is\ 1 \\ 0 & otherwise \end{cases}$$

- If at least one pixel under the SE is white → output = 1.
- If all are black → output = 0.

# Dilation

**Effect:**

- Fills small holes.
- Connects nearby white regions.
- Expands bright regions.

| Input | Structuring Element (3×3 all 1s) | Output (Dilated) |
|---|---|---|
| 0 0 0 0 0 | 1 1 1 | 0 1 1 1 0 |
| 0 1 1 1 0 | 1 1 1 | 1 1 1 1 1 |
| 0 1 1 1 0 | 1 1 1 | 0 1 1 1 0 |

**Result**: The “white” region grows larger.

# Example: Erosion

$$A = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$B = \mathbf{SE} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

| Position | Neighborhood under SE (3×3) | All 1? | Output pixel |
|---|---|---|---|
| (2,2) | $\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 1 \\ 0 & 1 & 1 \end{bmatrix}$ | No | 0 |
| (2,3) | $\begin{bmatrix} 0 & 0 & 0 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$ | No | 0 |
| (2,4) | $\begin{bmatrix} 0 & 0 & 0 \\ 1 & 1 & 0 \\ 1 & 1 & 0 \end{bmatrix}$ | No | 0 |
| (3,2) | $\begin{bmatrix} 0 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 1 & 1 \end{bmatrix}$ | No | 0 |
| (3,3) | $\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$ | Yes | 1 |

# Example: Erosion

$$A = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$B = \mathbf{SE} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

| Position | Neighborhood under SE (3×3) | All 1? | Output pixel |
|---|---|---|---|
| (3,4) | $\begin{bmatrix} 1 & 1 & 0 \\ 1 & 1 & 0 \\ 1 & 1 & 0 \end{bmatrix}$ | No | 0 |
| (4,2) | $\begin{bmatrix} 0 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 0 \end{bmatrix}$ | No | 0 |
| (4,3) | $\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 0 & 0 & 0 \end{bmatrix}$ | No | 0 |
| (4,4) | $\begin{bmatrix} 1 & 1 & 0 \\ 1 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$ | No | 0 |

Only the center pixel (3,3) remains 1.

# Example: Erosion

**Eroded image**

$$A \ominus B = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \ominus \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

The white region shrinks — only the inner core pixel remains.

# Example: Grayscale erosion

$$A = \begin{bmatrix} 30 & 40 & 35 \\ 50 & 60 & 55 \\ 45 & 70 & 65 \end{bmatrix} \qquad B = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

**Minimum value = 30**. So, g(2,2)=30

**Border Handling**

Assume zero-padding outside the image and output.

$$g = A \ominus B = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 30 & 40 & 35 & 0 \\ 0 & 50 & 60 & 55 & 0 \\ 0 & 45 & 70 & 65 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \ominus \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 30 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

# Example: Grayscale erosion

$$A = \begin{bmatrix} 30 & 40 & 35 \\ 50 & 60 & 55 \\ 45 & 70 & 65 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & \mathbf{30} & 40 & 35 & 0 \\ 0 & 50 & 60 & 55 & 0 \\ 0 & 45 & 70 & \mathbf{65} & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

| Position | Neighborhood (values) | Min |
|---|---|---|
| (1,1) | {0,0,0, 0,30,40, 0,50,60} | 0 |
| (1,2) | {0,0,0, 30,40,35,50, 60,55} | 0 |
| (1,3) | {0,0,0, 40,35, 0, 60,55,0} | 0 |
| (2,1) | {0, 30,40,0, 50, 60,0,45,70} | 0 |
| (2,2) | {30,40,35,50,60,55,45,70,65} | **30** |
| (2,3) | {40,35,0,60,55,0,70,65,0} | 0 |
| (3,1) | {0,50,60,0,45,70,0,0,0} | 0 |
| (3,2) | {50,60,55, 45,70,65, 0,0,0} | 0 |
| (3,3) | {60,55, 0,70, 65,0,0,0,0} | 0 |

# Example: Dilation

$$A \oplus B = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \oplus \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

| Position | Neighborhood (values) | At least any 1? | Output |
|---|---|---|---|
| (1,1) | {0,0,0, 0,1,1, 0,1,1} | Yes | 1 |
| (1,2) | {0,0,0, 1,1,1,1,1,1} | Yes | 1 |
| (1,3) | {0,0,0,1,1, 0, 1,1,0} | Yes | 1 |
| (2,1) | {0,1,1,0,1,1,0,1,1} | Yes | 1 |
| (2,2) | {1,1,1,1,1,1,1,1,1} | Yes | 1 |
| (2,3) | {1,1,0,1,1,0,1,1,0} | Yes | 1 |
| (3,1) | {0,1,1,0,1,1,0,0,0} | Yes | 1 |
| (3,2) | {1,1,1,1,1,1, 0,0,0} | Yes | 1 |
| (3,3) | {1,1, 0,1,1,0,0,0,0} | Yes | 1 |

# Example: Dilation

$$A = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & \mathbf{0} & \mathbf{1} & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & \mathbf{0} & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$B = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

| Position | Neighborhood (values) | At least any 1? | Output |
|---|---|---|---|
| (1,1) | {0,0,0,0,0,1,0,1,1} | Yes | 1 |
| (2,3) | {0,0,0,0,1,0,1,1,1} | Yes | 1 |
| (3,2) | {0,0,1,0,1,1,0,0,1} | Yes | 1 |
| (3,3) | {0,1,0,1,1,1, 0,1,0} | Yes | 1 |
| (3,4) | {1,0,0,1,1,0, 1,0,0} | Yes | 1 |
| (4,3) | {1,1,1,0,1,0,0, 0} | Yes | 1 |
| (4,4) | {1,1,0,1,0,0,0,0,0} | Yes | 1 |

$$A \oplus B = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \oplus \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

# Example: Dilation

$$A = \begin{bmatrix} 20 & 30 & 40 \\ 10 & 50 & 60 \\ 15 & 25 & 35 \end{bmatrix} \qquad B = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Output=$max(20, 30,40,10, 50, 60, 15, 25, 35) = 60.$

So, the **maximum center pixel becomes 60**.

$$\mathrm{A} = \begin{bmatrix} 20 & 30 & 40 \\ 10 & 50 & 60 \\ 15 & 25 & 35 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 20 & 30 & 40 & 0 \\ 0 & 10 & 50 & 60 & 0 \\ 0 & 15 & 25 & 35 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

If we do this for the whole image (ignoring borders or padding with zeros), the center region gets **brighter** — all pixels become the maximum in their neighborhood.

# Example: Dilation

$$A = \begin{bmatrix} 20 & 30 & 40 \\ 10 & 50 & 60 \\ 15 & 25 & 35 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & \mathbf{20} & 30 & 40 & 0 \\ 0 & 10 & \mathbf{50} & 60 & 0 \\ 0 & 15 & 25 & \mathbf{35} & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

| Position | Neighborhood (values) | Max |
|---|---|---|
| (1,1) | {0,0,0, 0,20,30, 0,10,50} | 50 |
| (1,2) | {0,0,0, 20,30,40, 10,50,60} | 60 |
| (1,3) | {0,0,0, 30,40, 0, 50,60,0} | 60 |
| (2,1) | {0, 20,30,0, 10, 50,0,15,25} | 50 |
| (2,2) | {20,30,40,10,50,60, 15, 25,35} | 60 |
| (2,3) | {30,40,0, 50,60,0, 25,35,0} | 60 |
| (3,1) | {0, 10,50,0, 15, 25,0,0,0} | 50 |
| (3,2) | {10,50,60, 15,25,35, 0,0,0} | 60 |
| (3,3) | {50,60, 0,25, 35,0,0,0,0} | 60 |

# Example: Dilation

$$A = \begin{bmatrix} 20 & 30 & 40 \\ 10 & 50 & 60 \\ 15 & 25 & 35 \end{bmatrix} \qquad B = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Output=$max(20, 30,40,10, 50, 60, 15, 25, 35) = 60.$

So, the **maximum center pixel becomes 60**.

$$g = A \oplus B = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 20 & 30 & 40 & 0 \\ 0 & 10 & 50 & 60 & 0 \\ 0 & 15 & 25 & 35 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \oplus \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 50 & 60 & 60 \\ 50 & 60 & 60 \\ 50 & 60 & 60 \end{bmatrix}$$

The bright area expanded — values increased because the max of each neighborhood dominates. Brightest regions "spread" to neighboring pixels. In this case, the brightest value **60** influenced most nearby pixels.

# Example: Dilation

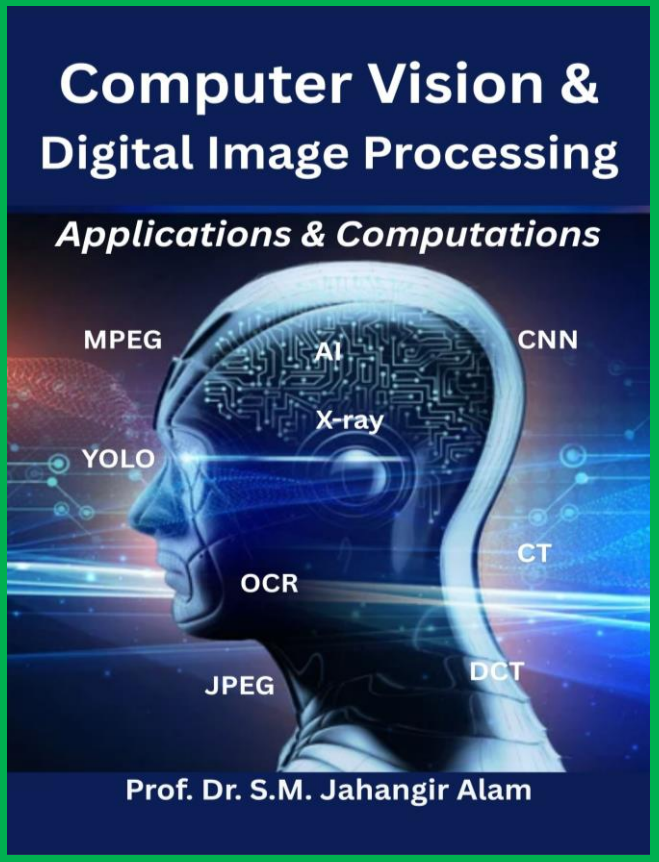
| Operation | Description | Effect |
|---|---|---|
| Grayscale Dilation | Replace each pixel by max in neighborhood | Expands bright regions, increases brightness, fills gaps |

| Operation | Binary Rule | Grayscale Rule | Effect |
|---|---|---|---|
| Erosion | Output=1 if all in SE=1 | Min filter | Shrinks bright regions |
| Dilation | Output=1 if any in SE=1 | Max filter | Expands bright regions |

Thank You

# Lecture 8.2

# Image Representation: Morphological Processing




Instructor Name:

**Prof. Dr. Jahangir Alam**

**Dept. of CSE, BAUST**

# Different Morphological Operations

**3. Opening**

- Opening is a compound operation that involves erosion followed by dilation.

- Purpose: To remove small objects or noise from the image while preserving the shape and size of larger objects.

- How it Works: First, the image undergoes erosion, which removes small objects and noise. Then, dilation is applied to restore the size of the remaining objects to their original dimensions.

# Different Morphological Operations

**4. Closing**

- Closing is another compound operation that consists of dilation followed by erosion.

- Purpose: To fill small holes and gaps in objects while preserving their overall shape.
- How it Works: First, dilation is applied to the image, filling small holes and gaps. Then, erosion is applied to restore the original size of the objects.

# Opening, Closing

Opening, Closing, and their combinations in morphological image processing.

These are built from erosion and dilation, so everything you learned before fits right in.

1. **Morphological Opening ($\circ$)**

2. **Morphological Closing ($\bullet$)**

# Opening, Closing

1. **Morphological Opening ($\circ$)**

Opening: **Erosion followed by Dilation**

$$A \circ B = (A \ominus B) \oplus B$$

**Intuition**

- **Erosion** first removes small white noise and shrinks objects.

- **Dilation** then restores the shape of remaining objects (without bringing back noise).

**Effect:** Removes small white spots / thin protrusions while preserving the main shape.

# Opening, Closing

## 1. Morphological Opening (∘)

$$B = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

**Binary Example:** Input image $A$:

$$A = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

| Position | Neighborhood (values) | All 1? | output |
|---|---|---|---|
| (1,1) | {0,0,0, 0,0,1, 0,1,1} | No | 0 |
| (2,2) | {0,1,0,1,1,1,0,1,0} | No | 0 |
| (3,3) | {1,1, 0,1,0,0,0,0,0} | No | 0 |

Structuring element $B$: 3×3 all 1s.

**Step 1: Erosion:** After erosion, small or thin connections vanish:

$$A \ominus B = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \ominus \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

# Opening, Closing

## 1. Morphological Opening (∘)

| Position | Neighborhood (values) | At least Any one 1? | Output |
|---|---|---|---|
| (1,1) | {0,0,0, 0,0,0, 0,0,0} | No | 0 |
| (2,2) | {0,0,0, 0,0,0, 0,0,0} | No | 0 |
| (3,3) | {0,0,0, 0,0,0, 0,0,0} | No | 0 |

**Step 2: Dilation:** Now dilate the eroded result:

$$(A \ominus B) \oplus B = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \oplus \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$A \circ B = (A \ominus B) \oplus B$$

**Result:** Small isolated noise is removed, object smoothed and slightly thinner.

# Opening, Closing

## 1. Morphological Closing (•)

**Closing:** Dilation followed by Erosion

$$A \cdot B = (A \oplus B) \ominus B$$

**Intuition**

- **Dilation** first expands bright regions, filling small holes.
- **Erosion** then shrinks back to original size — but holes remain filled.

**Effect:**

Fills small dark holes or gaps, connects broken regions.

# Opening, Closing

## 1. Morphological Closing (•)

$$A = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

(center has a "hole")

**Step 1: Dilation**

| Position | Neighborhood (values) | At least any 1? | Output |
|---|---|---|---|
| (1,1) | {0,0,0, 0,1,1, 0,1,0} | Yes | 1 |
| (2,2) | {1,1,1, 1,0,1, 1,1,1} | Yes | 1 |
| (3,3) | {0,1,0, 1,1,0, 0,0,0} | yes | 1 |

$$A \oplus B = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \oplus \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

# Opening, Closing

**1. Morphological Closing (•)**

**Step 2: Erosion**

| Position | Neighborhood (values) | All 1 | Output |
|---|---|---|---|
| (1,1) | {0,1,1, 1,1,1, 1,1,1} | No | 0 |
| (2,2) | {1,1,1, 1,1,1, 1,1,1} | Yes | 1 |
| (3,3) | {1,1,1, 1,1,1, 1,1,0} | No | 0 |

$$(A \oplus B) \ominus B = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix} \ominus \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$\boldsymbol{Closing}, A \cdot B = (A \oplus B) \ominus B = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Hole is filled, object more solid and connected. Fills small gaps and smooths the contour **from outside**.

# Opening, Closing

**Effect Comparison**

| Operation | Formula | Sequence | Main Effect |
|---|---|---|---|
| **Erosion** | $A \ominus B$ | — | Shrinks white regions |
| **Dilation** | $A \oplus B$ | — | Expands white regions |
| **Opening** | $(A \ominus B) \oplus B$ | Erosion → Dilation | Removes small white noise |
| **Closing** | $(A \oplus B) \ominus B$ | Dilation → Erosion | Fills small black holes |

# Example: Opening, Closing

**8.5##Grayscale** shows **both removing small bright spots** (via *opening*) and **filling small dark holes** (via *closing*) using real intensity values.

$$A = \begin{bmatrix} 10 & 20 & 30 & 20 & 10 \\ 20 & 80 & 25 & 80 & 20 \\ 30 & 25 & 15 & 25 & 30 \\ 20 & 80 & 25 & 80 & 20 \\ 10 & 20 & 30 & 20 & 10 \end{bmatrix}$$

$$B = SE = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

| Operation | Formula | Meaning |
|---|---|---|
| Erosion | $g(x,y) = \min_{(s,t) \in B} f(x+s, y+t)$ | Replaces each pixel with the minimum in its neighborhood |
| Dilation | $g(x,y) = \max_{(s,t) \in B} f(x+s, y+t)$ | Replaces each pixel with the maximum in its neighborhood |
| Opening | (A⊖B)⊕B | Removes small bright regions |
| Closing | (A⊕B)⊖B | Fills small dark regions |

# Example: Opening, Closing

**Erosion**

$$A_{erode} = \boldsymbol{A} \ominus \boldsymbol{B} = \begin{bmatrix} 10 & 20 & 30 & 20 & 10 \\ 20 & 80 & 25 & 80 & 20 \\ 30 & 25 & 15 & 25 & 30 \\ 20 & 80 & 25 & 80 & 20 \\ 10 & 20 & 30 & 20 & 10 \end{bmatrix} \ominus \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} 10 & 10 & 10 & 10 & 10 \\ 10 & 15 & 15 & 15 & 10 \\ 15 & 15 & 15 & 15 & 15 \\ 10 & 15 & 15 & 15 & 10 \\ 10 & 10 & 10 & 10 & 10 \end{bmatrix}$$

# Example: Opening, Closing

$$A_{open} = (\boldsymbol{A} \ominus \boldsymbol{B}) \oplus \boldsymbol{B} = \begin{bmatrix} 10 & 10 & 10 & 10 & 10 \\ 10 & 15 & 15 & 15 & 10 \\ 15 & 15 & 15 & 15 & 15 \\ 10 & 15 & 15 & 15 & 10 \\ 10 & 10 & 10 & 10 & 10 \end{bmatrix} \oplus \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} 15 & 15 & 15 & 15 & 15 \\ 15 & 15 & 15 & 15 & 15 \\ 15 & 15 & 15 & 15 & 15 \\ 15 & 15 & 15 & 15 & 15 \\ 15 & 15 & 15 & 15 & 15 \end{bmatrix}$$

**Effect:** the small bright 80s vanished — image is smoother and darker. Opening **removes tiny bright peaks (noise)**.

# Example: Opening, Closing

**Dilation**

$$\boldsymbol{A} \oplus \boldsymbol{B} = \begin{bmatrix} 10 & 20 & 30 & 20 & 10 \\ 20 & 80 & 25 & 80 & 20 \\ 30 & 25 & 15 & 25 & 30 \\ 20 & 80 & 25 & 80 & 20 \\ 10 & 20 & 30 & 20 & 10 \end{bmatrix} \oplus \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$$A_{dilate} = \boldsymbol{A} \oplus \boldsymbol{B} = \begin{bmatrix} 20 & 30 & 80 & 30 & 20 \\ 30 & 80 & 80 & 80 & 30 \\ 80 & 80 & 80 & 80 & 80 \\ 30 & 80 & 80 & 80 & 30 \\ 20 & 30 & 80 & 30 & 20 \end{bmatrix}$$

# Example: Opening, Closing

**Erosion**

$$(\boldsymbol{A} \oplus \boldsymbol{B}) \ominus \boldsymbol{B} = \begin{bmatrix} 20 & 30 & 80 & 30 & 20 \\ 30 & 80 & 80 & 80 & 30 \\ 80 & 80 & 80 & 80 & 80 \\ 30 & 80 & 80 & 80 & 30 \\ 20 & 30 & 80 & 30 & 20 \end{bmatrix} \ominus \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$$A_{close} = (\boldsymbol{A} \oplus \boldsymbol{B}) \ominus \boldsymbol{B} = \begin{bmatrix} 20 & 30 & 30 & 30 & 20 \\ 30 & 80 & 80 & 80 & 30 \\ 30 & 80 & 80 & 80 & 30 \\ 30 & 80 & 80 & 80 & 30 \\ 20 & 30 & 30 & 30 & 20 \end{bmatrix}$$

**Effect:** the **dark hole (15)** is now filled (replaced by ~80); edges are smoother — small gaps or holes vanish.

# Hit-or-Miss Transform

**5. Hit-or-Miss Transform**

- The hit-or-miss transform is used to find specific patterns or shapes in a binary image.

- Purpose: To detect specific configurations or shapes in the image.

- How it Works: It uses a pair of structuring elements: one for the foreground and one for the background. The operation looks for places where the foreground structuring element matches the foreground in the image, and the background structuring element matches the background in the image.

# Hit-or-Miss Transform

- The **Hit-or-Miss Transform** is used to **detect specific patterns** or shapes in a binary image.

- Given a binary image $A$ and a structuring element $B$, the HMT looks for **exact matches** of a shape.

- Usually, $B$ is divided into **two disjoint parts**:
  - $B_1$ — the foreground (must match 1s)
  - $B_2$ — the background (must match 0s)

- **Mathematical form:**

$$HMT(A, B) = (A \ominus B_1) \cap (A^c \ominus B_2)$$

# Hit-or-Miss Transform

**Key Points**

- **HMT is for pattern detection**, not general shape smoothing.
- Works **only for binary images**.
- Can detect **corners, endpoints, T-junctions**, or any small predefined pattern.
- Very sensitive: the pattern must **match exactly**, otherwise it won’t detect anything.

# Hit-or-Miss Transform

- $A = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}, B = \begin{bmatrix} -1 & 1 & -1 \\ 1 & 1 & 1 \\ -1 & 1 & -1 \end{bmatrix},$

- $B_1 = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 0 \end{bmatrix}, B_2 = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 0 & 0 \\ 1 & 0 & 1 \end{bmatrix}$

- $A \ominus B_1 = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \ominus \begin{bmatrix} 0 & 1 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$

# Hit-or-Miss Transform

- $A = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \Rightarrow A^c = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$

- $A^c \ominus B_2 = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix} \ominus \begin{bmatrix} 1 & 0 & 1 \\ 0 & 0 & 0 \\ 1 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$

# Hit-or-Miss Transform

- $HMT(A, B) = (A \ominus B_1) \cap (A^c \ominus B_2)$

$$= \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix} \cap \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

# Morphological Gradient

- **6. Morphological Gradient**
- The morphological gradient is the difference between the dilation and erosion of an image.

- Purpose: To highlight the boundaries or edges of objects in the image.

- How it Works: By subtracting the eroded image from the dilated image, the boundaries of the objects are emphasized.

# Morphological Gradient

- The **Morphological Gradient** is a fundamental operation in **mathematical morphology** used to **highlight the edges** or boundaries of objects in a binary or grayscale image. Let's break it down clearly.

- The **Morphological Gradient** $\boldsymbol{G}$ of an image $A$ using a structuring element $B$ is defined as:

$$G = (A \oplus B) - (A \ominus B)$$

Where:

- $A \oplus B$ = **dilation** of $A$ by $B$
- $A \ominus B$ = **erosion** of $A$ by $B$
- The subtraction is **element-wise**

# Morphological Gradient

**Intuition**

- **Dilation** expands the object → highlights **outer boundary**
- **Erosion** shrinks the object → highlights **inner boundary**
- **Difference** → leaves only the **edges of the object.**

$$\textbf{\textit{Morphological Gradient} = \textit{Boundary of the object}}$$

# Morphological Gradient

- $A = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix}, B = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 0 \end{bmatrix}$

$$A \oplus B = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix} \oplus \begin{bmatrix} 0 & 1 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 0 \end{bmatrix}$$

$$A \ominus B = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix} \ominus \begin{bmatrix} 0 & 1 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

# Morphological Gradient

$$G = (A \oplus B) - (A \ominus B)$$

$$= \begin{bmatrix} 0 & 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 0 \end{bmatrix} - \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 1 & 1 \\ 0 & 1 & 1 & 1 & 0 \end{bmatrix}$$

# Top-Hat Transform

**7. Top-Hat Transform**

- The top-hat transform is used to extract small elements and details from an image.

- Purpose: To enhance bright objects on a dark background or vice versa.

- How it Works: There are two types: white top-hat (original image minus the result of opening) and black top-hat (result of closing minus the original image).

# Top-Hat Transform

The **Top-Hat Transform** is a morphological operation used to **extract small elements or details** from an image that are **brighter than their surroundings**.

- There are two types of Top-Hat transforms:

**a) White Top-Hat (WTH) – detects bright elements**

$$WTH(A) = A - (A \circ B)$$

**b) Black Top-Hat (BTH) – detects dark elements**

$$BTH(A) = (A \cdot B) - A$$

# Top-Hat Transform

- $A = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix}, B = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 0 \end{bmatrix}$

- $A \ominus B = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix} \ominus \begin{bmatrix} 0 & 1 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$

# Top-Hat Transform

- $(A \ominus B) \oplus B = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \oplus \begin{bmatrix} 0 & 1 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 0 \end{bmatrix}$

$$= \begin{bmatrix} 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$

# Top-Hat Transform

**a) White Top-Hat (WTH) – detects bright elements**

$$WTH(A) = A - (A \circ B) = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix} - \begin{bmatrix} 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$

$$= \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

# Top-Hat Transform

- **b) Black Top-Hat (BTH) – detects dark elements**

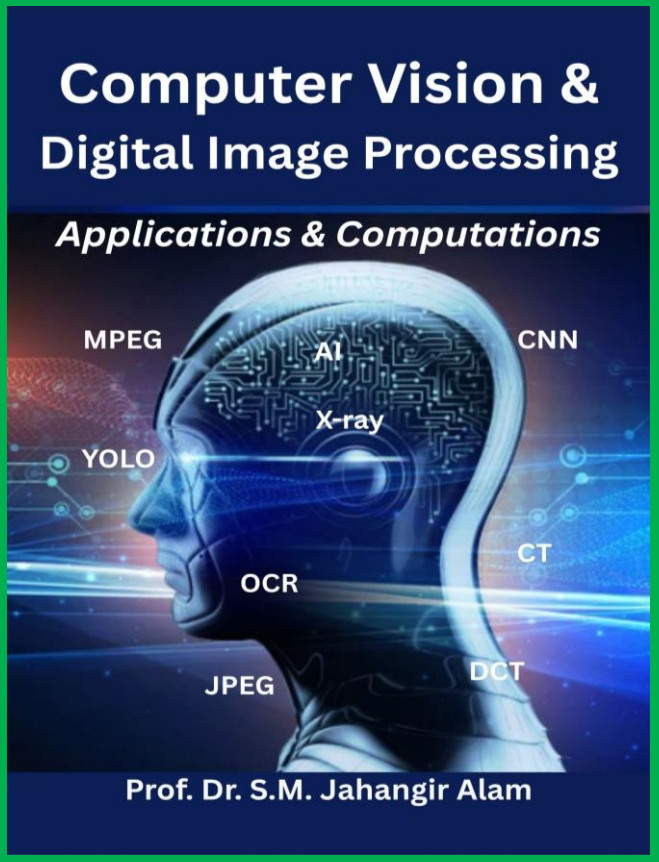
$$BTH(A) = (A \circ B) - A = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix} - \begin{bmatrix} 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$

$$= \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Thank You

# Lecture 8.3

# Image Representation: Morphological Processing




Instructor Name:

**Prof. Dr. Jahangir Alam**

**Dept. of CSE, BAUST**

# Skeletonization

**8. Skeletonization**

- Skeletonization reduces objects in a binary image to their skeletal form.

- Purpose: To simplify objects to their essential structure while preserving their connectivity.

- How it Works: It iteratively erodes the image until the objects are reduced to a minimal, skeletal form.

# Pruning

**9. Pruning**

- Pruning removes small spurs or extraneous branches from the skeleton of an image.

- Purpose: To clean up the skeleton by removing unwanted artifacts.

- How it Works: It identifies and removes small, irrelevant branches from the skeletonized image.

# Skeletonization

- Skeletonization, one of the most important operations in morphological image processing, often used for shape analysis, object representation, and thinning of binary images.
- Skeletonization (also called morphological thinning) is a process that reduces binary objects to their minimal, one-pixel-wide representations, while preserving their topology (connectivity and structure).
- It’s like stripping away layers of pixels from the edges until only the “center lines” (skeleton) remain.

# Skeletonization

Given a binary image $A$, the **skeleton** $S(A)$ is defined as the union of the "skeleton subsets":

$$S(A) = \bigcup_{k=0}^{K} S_k(A)$$

where

$$S_k(A) = (A \ominus kB) - [(A \ominus kB) \circ B]$$

Here:

- $\ominus$: **Erosion**
- $\circ$: **Opening** (erosion followed by dilation)
- $B$: **Structuring element**
- $k$: Number of erosions applied

# Skeletonization

**In simple terms:**

- Erode the image repeatedly.
- After each erosion, perform an opening to remove parts that can’t fit the structuring element.
- Subtract the opened image from the eroded image — that gives one skeleton layer.
- Continue until the image disappears.

# Skeletonization: Applications

- **OCR (Optical Character Recognition)** – extracting character strokes
- **Fingerprint recognition** – ridge thinning
- **Medical imaging** – blood vessel or neuron tracing
- **Shape analysis** – central axes for object comparison
- **Path planning** – medial axis for navigation

**Summary:**

1. Repeatedly erode the object.
2. Open each eroded image to remove protrusions.
3. Subtract opened image from eroded image → skeleton layer.
4. Combine layers → final skeleton.

# Skeletonization: Algorithm

**Step 1: Input:** Binary image $A$, 1 for object pixels, 0 for background pixel.

**Step 2: Initialize:** Set $k = 0$ and initialize the skeleton $S = \emptyset$.

**Step 3: Iterative Erosion 1,** $A_{k+1} = A \ominus kB \Rightarrow A_1 = \boldsymbol{A \ominus B}$

Perform morphological erosion repeatedly using a structuring element $B$(e.g., 3×3 square). That is, erode the image $A$$k$times.

**Step 4: Opening 1,** $A_{k+1} \circ \boldsymbol{B} = (A_{k+1} \ominus \boldsymbol{B}) \oplus \boldsymbol{B}$

$$\Rightarrow A_1 \circ \boldsymbol{B} = (A_1 \ominus \boldsymbol{B}) \oplus \boldsymbol{B}$$

This removes small protrusions that disappear after erosion.

# Skeletonization: Algorithm

**Step 5: Skeleton Layer,** $S_{k+1} = A_{k+1} - (A_{k+1} \circ \boldsymbol{B})$

$$\Rightarrow S_1 = A_1 - (A_1 \circ \boldsymbol{B})$$

Extract the difference between successive erosions. Each $S_k$ gives a thin contour layer of the skeleton.

**Step 6: Repeat**

- If $K$ is the last iteration before the object disappears completely after erosion. Continue with $A_1, A_2, \ldots$ until all pixels are removed. Finally, combine all skeleton layers:

$$S(A) = S_1 \cup S_2 \cup \cdots = S(A) = \bigcup_{k=1}^{K} S_k(A)$$

- **Step 7: Output**
- The final skeleton $S$ represents the **medial axis** — thin, connected lines that preserve the object's topology and length.

# Skeletonization: Algorithm

**Step 1: Input:** Binary image $A$, 1 for object pixels, 0 for background pixel.

**Step 2: Initialize:** Set $k = 0$ and initialize the skeleton $S = \emptyset$.

**Step 3: Iterative Erosion 1,** $A_{k+1} = A \ominus kB \Rightarrow A_1 = \boldsymbol{A \ominus B}$

**Step 2: Opening 1,** $A_{k+1} \circ \boldsymbol{B} = (A_{k+1} \ominus \boldsymbol{B}) \oplus \boldsymbol{B}$

$$\Rightarrow A_1 \circ \boldsymbol{B} = (A_1 \ominus \boldsymbol{B}) \oplus \boldsymbol{B}$$

**Step 3: Skeleton Layer,** $S_{k+1} = A_{k+1} - (A_{k+1} \circ \boldsymbol{B})$

$$\Rightarrow S_1 = A_1 - (A_1 \circ \boldsymbol{B})$$

**Step 4: Repeat:** $S(A) = S_1 \cup S_2 \cup \cdots$

**Step 5: Erosion 2:** $A_{k+1} = A_k \ominus \boldsymbol{B} \Rightarrow A_2 = A_1 \ominus \boldsymbol{B}$

**Step 6: Opening 2:** $\boldsymbol{A_2} \circ \boldsymbol{B} = (A_1 \ominus \boldsymbol{B}) \oplus \boldsymbol{B}$

**Step 7: Skeleton Layer 1:** $S_2 = \boldsymbol{A_2} - (\boldsymbol{A_2} \circ \boldsymbol{B})$

**Step 8: Combine Skeleton Layers**

$$S(A) = S_1 \cup S_2$$

# Skeletonization: Example1

$$A = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \qquad B = SE = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

**Step 1: Erosion 1:** $A_1 = \boldsymbol{A} \ominus \boldsymbol{B}$

This removes outer layers. The result:

$$A_1 = \boldsymbol{A} \ominus \boldsymbol{B} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \ominus \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

# Skeletonization

**Step 2: Opening 1:** $A_1 \circ \boldsymbol{B} = (A_1 \ominus \boldsymbol{B}) \oplus \boldsymbol{B}$

$$A_1 \ominus \boldsymbol{B} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \ominus \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

**Dilate**

$$A_1 \circ \boldsymbol{B} = (A_1 \ominus \boldsymbol{B}) \oplus \boldsymbol{B} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \oplus \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

# Skeletonization

- **Step 3: Skeleton Layer** $1$: $S_1 = A_1 - (A_1 \circ \boldsymbol{B})$
- This isolates edge differences — the first skeleton layer appears inside.

$$S_1 = A_1 - (A_1 \circ \boldsymbol{B})$$

$$= \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} - \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$= \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

(negative values → 0 in binary morphology)

**Step 4: Repeat**

Continue with $A_1, A_2, \ldots$ until all pixels are removed.

Finally, combine all skeleton layers:

$$S(A) = S_1 \cup S_2 \cup \cdots$$

# Skeletonization

**Step 5: Erosion 2 (k=2):** $A_2 = A_1 \ominus \boldsymbol{B}$

Erode $A_1$ again:

$$A_2 = A_1 \ominus \boldsymbol{B} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \ominus \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

**Step 6: Opening 2:** $\boldsymbol{A_2 \circ B = (A_2 \ominus B) \oplus B}$

$$\boldsymbol{A_2 \ominus B} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \ominus \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

# Skeletonization

**Step 6: Opening 2:** $A_2 \circ B = (A_2 \ominus B) \oplus B$

$$A_2 \ominus B = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \ominus \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$A_2 \circ B = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \oplus \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

# Skeletonization

**Step 7: Skeleton Layer 2:** $S_2 = \boldsymbol{A_2} - (\boldsymbol{A_2} \circ \boldsymbol{B})$

$$S_2 = \boldsymbol{A_2} - (\boldsymbol{A_2} \circ \boldsymbol{B}) = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} - \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$= \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

**Step 8: Combine Skeleton Layers**

$$S(A) = S_1 \cup S_2 = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \cup \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

# Remove Small Objects or Fill Holes

$$A = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}, B = SE = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

**1. Remove Small Objects → Morphological Opening**

$$A \ominus B = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \ominus \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

# Remove Small Objects or Fill Holes

**1. Remove Small Objects → Morphological Opening**

$$A \ominus B = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \ominus \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$A_{open} = (\boldsymbol{A} \ominus \boldsymbol{B}) \oplus \boldsymbol{B} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \oplus \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

**Small object (2,4)** is gone — only the main region remains and is slightly smoother.

# Remove Small Objects or Fill Holes

**2. Fill Holes → Morphological Closing**

$$A \oplus B = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \oplus \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

$$A_{close} = (\boldsymbol{A} \oplus \boldsymbol{B}) \ominus \boldsymbol{B} = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix} \ominus \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

The **hole at the center (3,3)** is now **filled**.

# Lung Cancer Detection using Transfer Learning

**8.20##Lung Cancer Detection using Transfer Learning, to understand the main ideas without complex computation.**

**Objective:**

- Classify lung CT images as **Cancerous (1)** or **Non-Cancerous (0)** by leveraging a pretrained CNN model (e.g., ResNet, VGG) and fine-tuning it for lung cancer detection.

**Step 1: Pretrained Model (Feature Extractor)**

- Assume you have a pretrained CNN on ImageNet (trained to recognize 1000 classes of everyday objects).
- The pretrained model extracts **general image features** (edges, textures, shapes) from input images.

# Lung Cancer Detection using Transfer Learning

**Step 2: Input Lung CT Image (Simplified)**

Suppose the input CT image (grayscale, resized to 4×4 for example):

| | | | |
|---|---|---|---|
| 10 | 12 | 15 | 14 |
| 60 | 80 | 58 | 14 |
| 75 | 90 | 70 | 16 |
| 55 | 65 | 50 | 15 |

**Step 3: Feature Extraction by Pretrained CNN**

- Instead of training from scratch, **pass the CT image through the pretrained CNN** layers up to a certain layer.
- The CNN outputs a feature vector representing the input image.

For simplicity, suppose the pretrained CNN outputs this 3-dimensional feature vector:

$$f = [0.75, 0.6, 0.9]$$

# Lung Cancer Detection using Transfer Learning

- **Step 4: Add Custom Classifier Layer**
- Add a small neural network layer on top of pretrained features for lung cancer classification.
- For example, a simple logistic regression (one fully connected layer with sigmoid activation):

$$y = \sigma(w_1 f_1 + w_2 f_2 + w_3 f_3 + b)$$

Where,

- $w = [2.0, -1.0, 1.5]$ are weights (learned during fine-tuning)
- $b = -0.5$ is bias
- $\sigma$ is sigmoid activation function

# Lung Cancer Detection using Transfer Learning

**Step 5: Compute Output (Probability)**

- Calculate:

$$z = 2.0 \times 0.75 + (-1.0) \times 0.6 + 1.5 \times 0.9 - 0.5$$
$$= 1.5 - 0.6 + 1.35 - 0.5 = 1.75$$

- Sigmoid:

$$y = \frac{1}{1 + e^{-1.75}} \approx 0.85$$

**Step 6: Decision**

- Since $y = 0.85 > 0.5$, predict **Cancerous (1)**.

# Lung Cancer Detection using Transfer Learning

| Step | Operation | Result |
|---|---|---|
| **Input** | Lung CT Image (4×4 pixels) | Pixel values |
| **Pretrained CNN Feature Extraction** | Output 3D feature vector | [0.75, 0.6, 0.9] |
| **Custom Classifier** | Logistic regression on features | $z$=1.75, $y$=0.85 |
| **Classification** | Sigmoid threshold | Cancerous (1) |

# Facial Expression Detection using Deepface Module

**8.23##Illustrate how facial expression detection works conceptually using a tool like the DeepFace module, focusing on the core flow without actual coding or heavy math.**

**Objective:**

- Given a face image, detect the person's **facial expression** (e.g., happy, sad, angry).

**Step 1: Input Face Image**

Imagine a grayscale 5×5 pixel face patch (very simplified):

| | | | | |
|---|---|---|---|---|
| 120 | 130 | 140 | 130 | 120 |
| 125 | 150 | 170 | 150 | 125 |
| 130 | 160 | 200 | 160 | 130 |
| 125 | 150 | 170 | 150 | 125 |
| 120 | 130 | 140 | 130 | 120 |

Center pixels are brighter (e.g., indicating eyes and nose).

# Facial Expression Detection using Deepface Module

**Step 2: Face Detection and Alignment (Simplified)**

- Detect face region → already cropped here.
- Align face (normalize orientation) → assume already aligned.

**Step 3: Feature Extraction (Using DeepFace pre-trained model)**

- Extract deep features representing facial details.
- Suppose extracted feature vector (simplified) is:

$$\boldsymbol{f} = [0.75, 0.8, 0.2, 0.1, 0.05]$$

Each dimension corresponds to abstract facial attributes (like muscle movements).

# Facial Expression Detection using Deepface Module

**Step 4: Expression Classification**

•Use a classifier (e.g., softmax layer) trained on expressions:

| Expression | Weight Vector w |
|---|---|
| Happy | [2, 1, 0, 0, 0] |
| Sad | [0, 1, 2, 1, 1] |
| Angry | [1, 0, 1, 2, 1] |

Calculate score for each expression:

$$s = \boldsymbol{w} \cdot \boldsymbol{f} = \sum w_i f_i$$

- **Happy:** $2 * 0.75 + 1 * 0.8 + 0 + 0 + 0 = 1.5 + 0.8 = 2.3$
- **Sad:** $0 + 1 * 0.8 + 2 * 0.2 + 1 * 0.1 + 1 * 0.05 = 0.8 + 0.4 + 0.1 + 0.05 = 1.35$
- **Angry:** $1 * 0.75 + 0 + 1 * 0.2 + 2 * 0.1 + 1 * 0.05 = 0.75 + 0 + 0.2 + 0.2 + 0.05 = 1.2$

# Facial Expression Detection using Deepface Module

**Step 5: Final Decision**

- Highest score = **Happy (2.3)**
- Predict facial expression: **Happy**

| Step | Operation | Result |
|---|---|---|
| **Input** | 5×5 grayscale face image | Pixel intensities |
| **Feature Extraction** | DeepFace pre-trained features | [0.75, 0.8, 0.2, 0.1, 0.05] |
| **Classification Scores** | Dot product with weight vectors | Happy=2.3, Sad=1.35, Angry=1.2 |
| **Prediction** | Max score | Happy |

Thank You